Caméléon

2011 Wiki Quick Dump

Caméléon[1]

2011 Wiki Quick Dump

[1] Caméléon
http://fr.wikipedia.org/wiki/cameleon

Auteurs

Rôle	Prénom, Nom	Rôle, Structure
Rédacteur	Olivier Cugnon de Sévricourt	Architecte, Shinoe
Rédacteur	Vincent Tariel	Directeur, Alpha Numeric Vision

Intention

Ce document est une reprise des informations présentent sur le site internet http://cameleon.shinoe.org avant sa fermeture pour fin de maintenance.

Nous voulions conserver ces informations car elles nous permettent d'identifier les positions que nous avions à l'époque (2011) au regarde du langage Caméléon.

Méthodologie

De simple copier/coller avec une modification de style.

Structure

La structure reprends la structure du menu de gauche du wiki Caméléon (http://cameleon.shinoe.org).

Notes de versions

Version	Description	Date	Auteur
0.1	Création à partir du wiki cameleon.shinoe.org (daté de 2011)	09/10/2014	Olivier CDS
0.2	Revue de style	16/10/2014	Olivier CDS

Table des matières

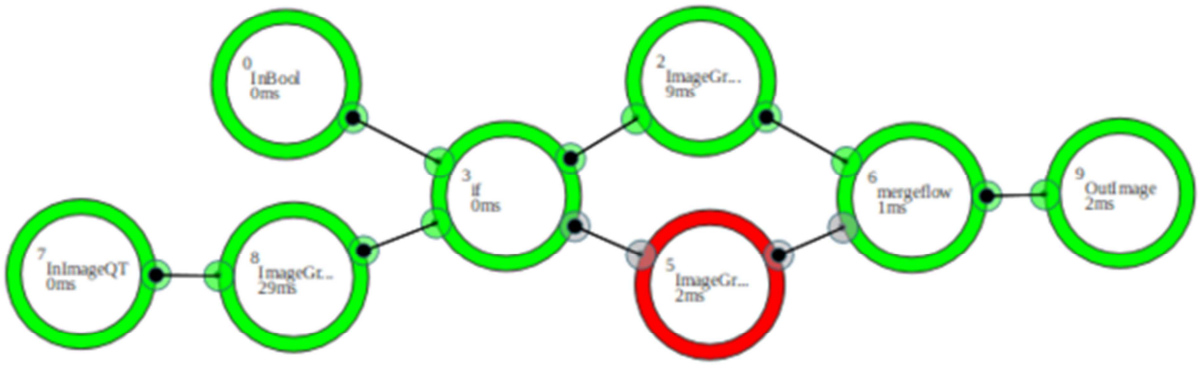
Caméléon 2
Auteurs 3
Intention 4
Méthodologie 4
Structure 4
Notes de versions 5
Table des matières 6
Introduction 9
A simple view 12
  Cameleon Language 12
  Community vision 21
  The end 24
About 25
  Shinoe 25
  Philosophy & Ambition 25
  Support team! 25
  What's NeXt ?! 25
  We are looking for hands & minds! 25
  Beta 26
  Licence 26
  Limitation 26
  Dictionaries 26
  First Release & First Source Release 26
First Step 27
  First Launch 27
  Workspace 27
  Basics 28
    operator & dictionary 28
    connection 29
    dictionary 29
    composition & composer 30
  Hello world! 30
    A simple addition 30
    First Operator 31

First Connections 32
Manage the view: move & zoom 32
Data Editor & Viewer 32
Launch your beautiful work 34
Save, Build & launch your work as a standalone application 35
See Also 35
Documentation 36
Introduction 36
Features list 36
Basics 37
Workspace 39
Creation 39
Select & Launch 41
Composer 41
Workspace 41
Menu Bar 43
File Menu 43
Tools Menu 43
Help Menu 44
Player Bar 44
Operator's Factory 45
Graph Editor 45
Create & delete Operator 45
Move an Operator 46
Connect & disconnect Operator 46
Manage the view: move & zoom 47
Clear & properties 47
Operator 47
Connector Colors & statuses 48
Operator Colors & statuses 48
Information tooltips 49
Invisible connector 50
Execution condition 51
Composition 51
Controller 51

Selector bar 51
View 52
Editor 52
Operator Graphic 53
Manage dictionaries 53
Install a new dictionary 53
Update an existing dictionary 56
Build app 56
See Also 57
High level bottom-up 58
Contents 58
Multi-scale programming 58
Top-down 58
Bottom-up 59
Low-level 59
High-level 59
Mix 59
Cameleon, a framework to design high-level bottom-up languages 60
Some hypothetical use cases 60
annexes 61
quoted[2] 61

Introduction

Cameleon is a new graphical language for functional programming

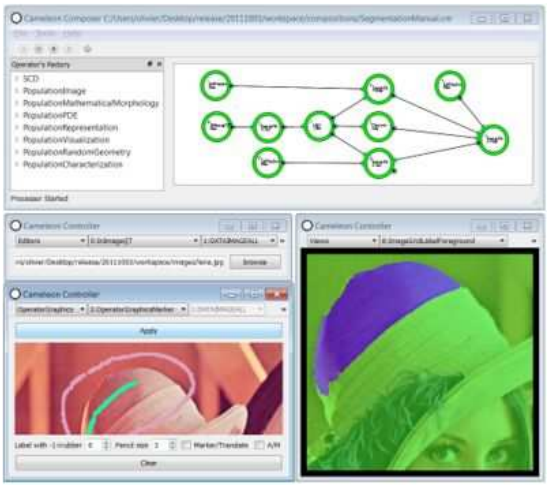


THIS IS THE OLD BETA 1 DOCUMENTATION WEB SITE. This web site is deprecated.

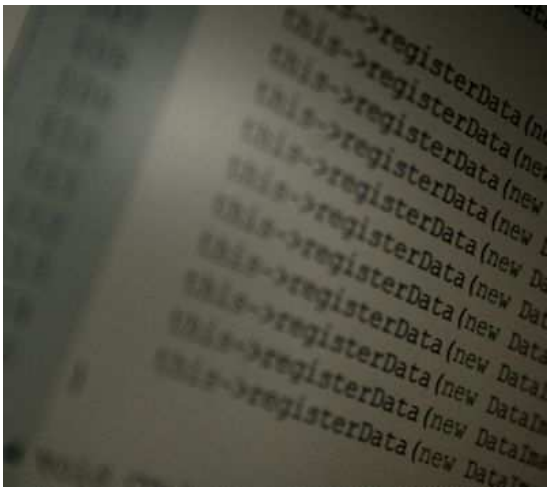
*The current Cameleon version is in a **beta** state. The open beta 2 is available here!*

Contact-us if you have any question.

Cameleon is a new free language based on an abstract functional grammar & a glossary divided in specifics working fields dictionaries. The grammar is based on a petri net extension with a mathematical formalism described in Cameleon grammar, a petri net extension. The language can be edited graphically using the **Cameleon Integrated Development Environment (IDE)** to create Cameleon program, named composition. The language glossary can be extended with new fields dictionaries using the **Cameleon Dictionary Development Kit (DDK)** to create Cameleon dictionary, a collection of words, them-self named operators. Cameleon language aims to simplify & accelerate software development by using a high-level bottom-up approach.



Cameleon IDE, segmentation composition with composer & controllers



DDK, data registration with the integration framework

Cameleon IDE is composed by:

- Interpreter, a Cameleon Language interpreter with:
  - *Real Time Updater*: you update your composition, the interpreter takes care of your change during the execution,
  - *Dictionaries installer:* you install 3rd party dictionary integrated thanks to Cameleon DDK,
- Composer, a graph editor with:
  - *Data process Composer:* you code Cameleon program (named composition) with simple drag & drop,
  - *Real Time Processor:* you see the inner execution of your program in real time,
- Controller, a GUI editor with:
  - *Data Controller:* you create GUI including data editors/viewers with simple drag & drop,
  - *Real Time GUI:* you see & manipulate data in real time,
- Builder, an application builder with:
  - *Standalone Application Builder:* you build & deploy your application with a single click,
- Dictionaries available now on shinoe plateform:
  - *Standard Cameleon Dictionary:* a dictionary dedicated to number, boolean, string & table data,
  - *Generics Dictionary:* a dictionary dedicated to execution loop pattern & conditional execution,
  - *Population Dictionary:* a dictionary dedicated to image processing & analysis.

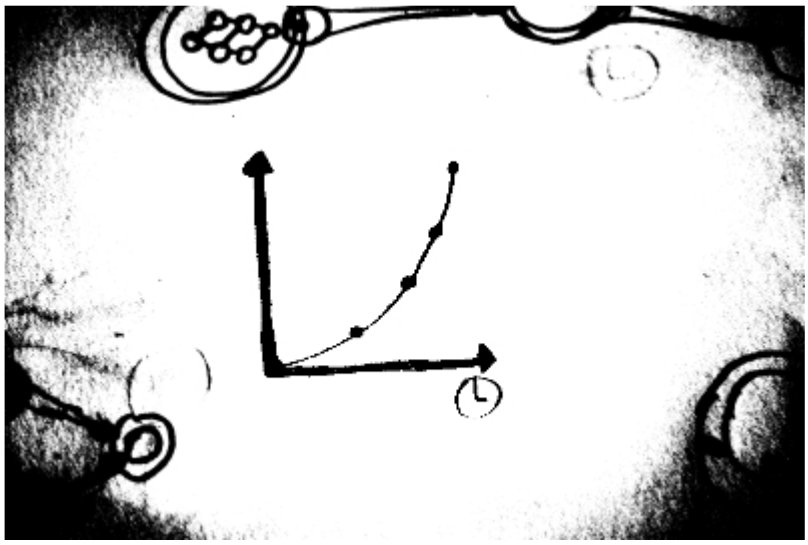
Cameleon DDK (available soon!) allows:

- C++ library integration framework with:
  - *Data:* integrate your own data structures,
  - *Operator :* integrate your own algorithms,
  - *View:* integrate your own data viewers,
  - *Editor:* integrate your own data editors.
- Accelerated quality control tools with:
  - *Analysis tools:* performance probe & bench marking,
  - *Operator Unit Testing:* automatics unit testing,
  - *Non Regression Testing:* automatics non regression testing.
- Distributed execution tools with:
  - *Client composer:* composition is viewed & edited from a local computer
  - *Server interpreter:* composition is executed on a distant computer
  - *Ressource manager:* data network synchronizer
- Parallel execution tools with:
  - *Parallel interpreter:* parallel composition computing
  - *Parallel analysis:* parallel composition analyzing
  - *Parallel API:* parallel operator computing

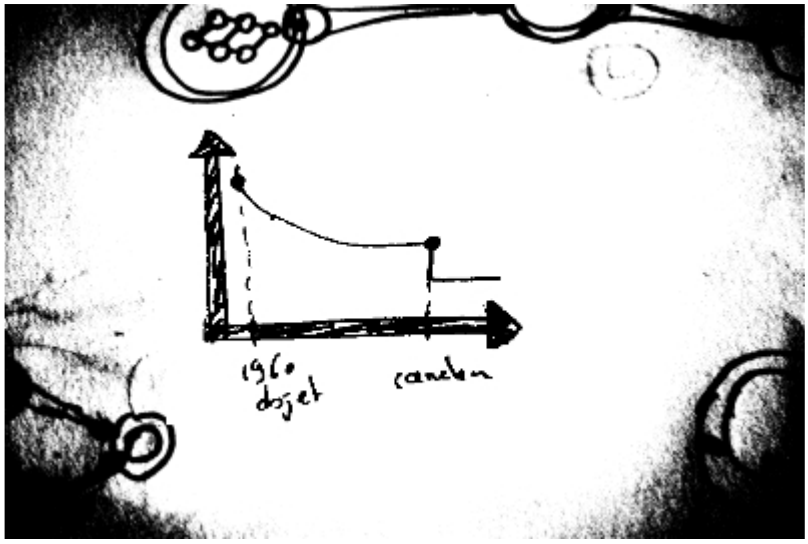
A simple view

This little storyboard is an essai to explain simply what is Cameleon & where goes our vision. It makes some shortcuts to make the whole things understandable by non technical profiles.

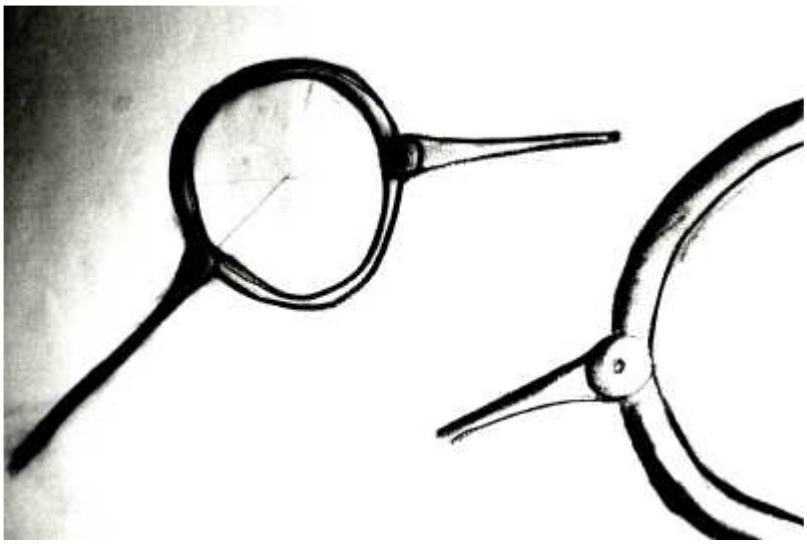
Cameleon Language



Our world is evolving. Problems complexity growth ...



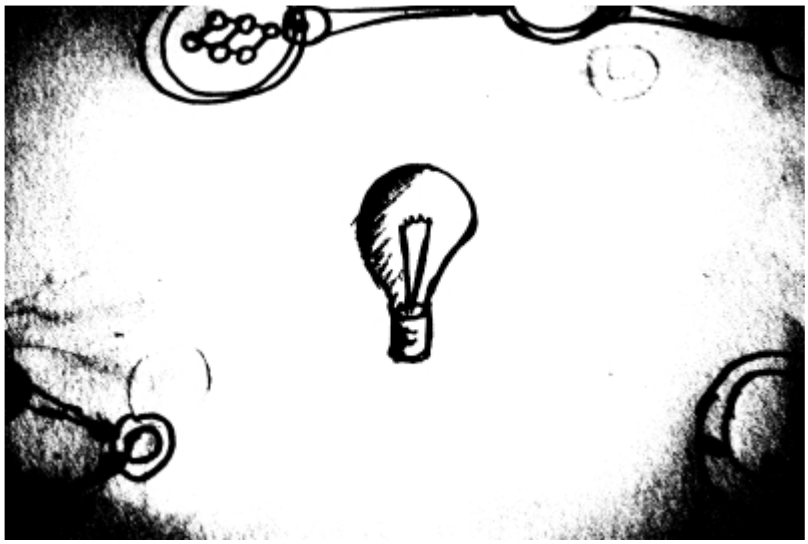
... when programming language complexity is quite the same since the emergence of the object programming paradigm (1960).



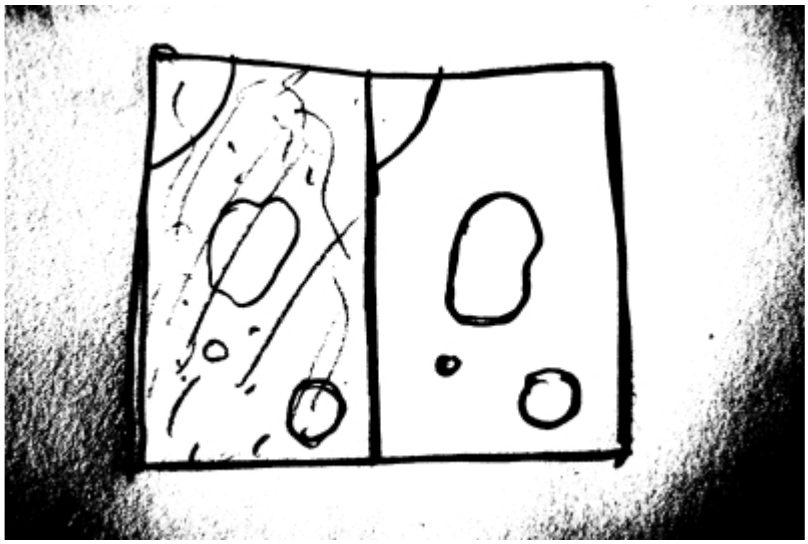
Cameleon is our response to complexity. Our goal is to share a solution to simplify data processing creation & software development. This solution is composed by a new programming language, an integrated development environment & a development kit.



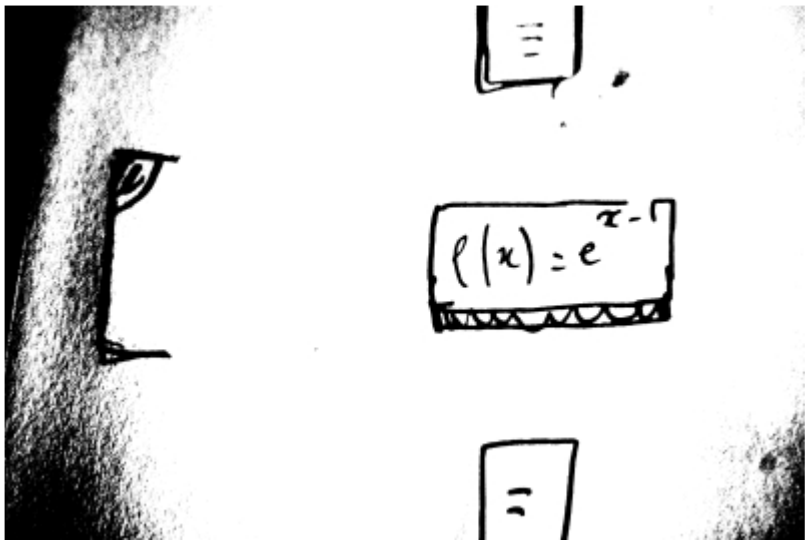
You have a problem ?



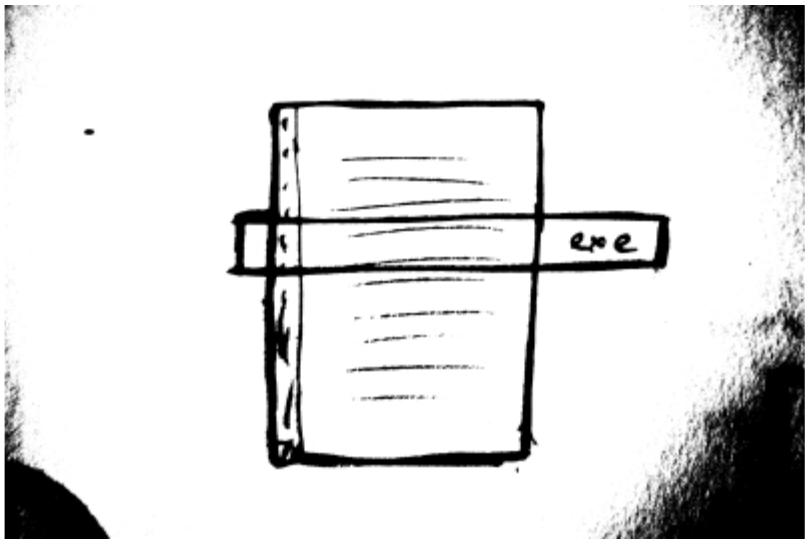
And already an idea to solve it with a software development technology and a standard method



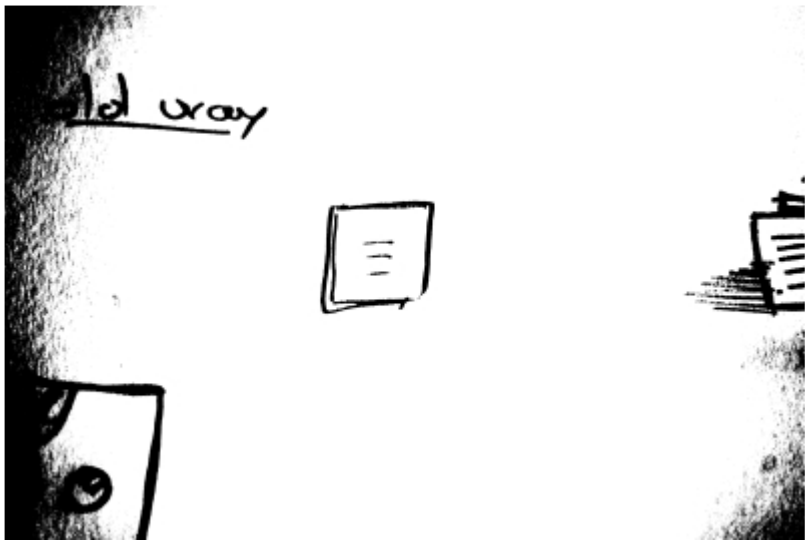
From this idea you design an application composed with a data-process, GUI drawings ...



... and a core engine already modeled. You have designed your application at a macro level & you seems to be ready to implement your idea.



You choose a written programming language (C++, Python, Java, .Net...) We call those micro languages



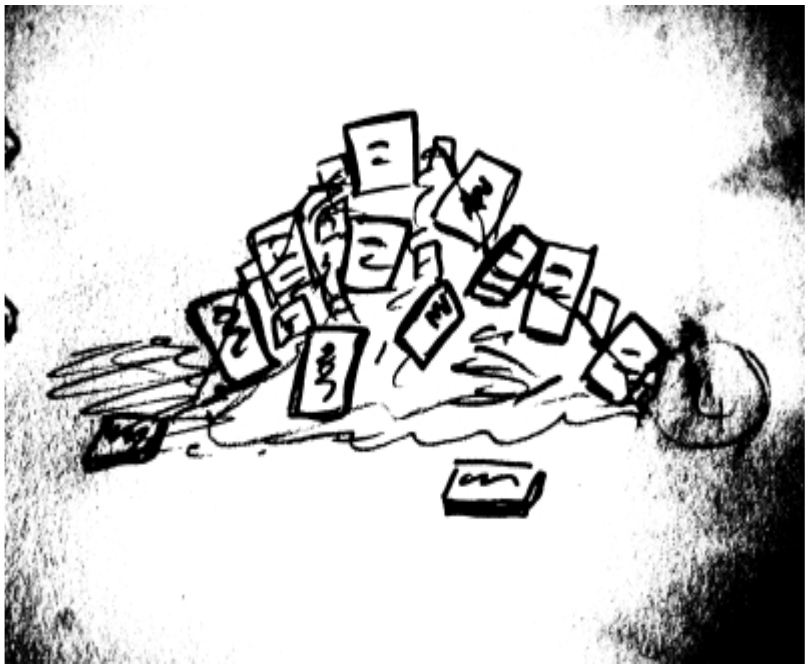
You start, with your team, to write a page of code.



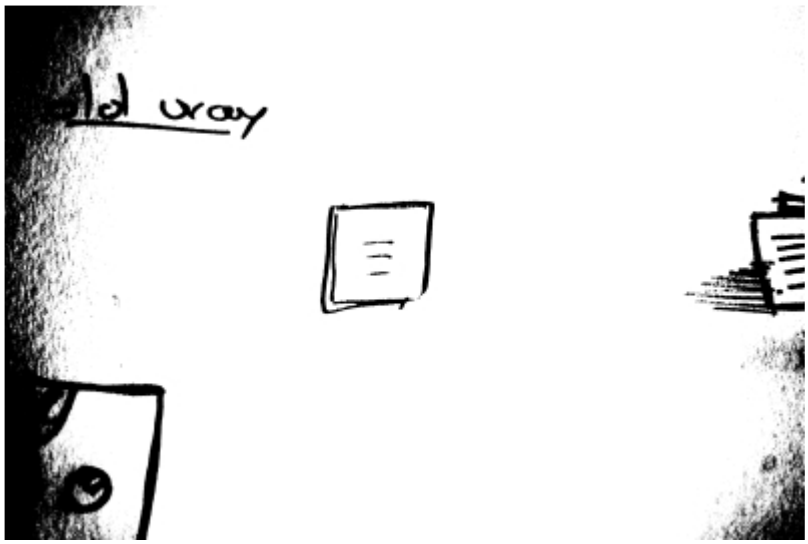
With time, this page of code becomes a large amount of code.



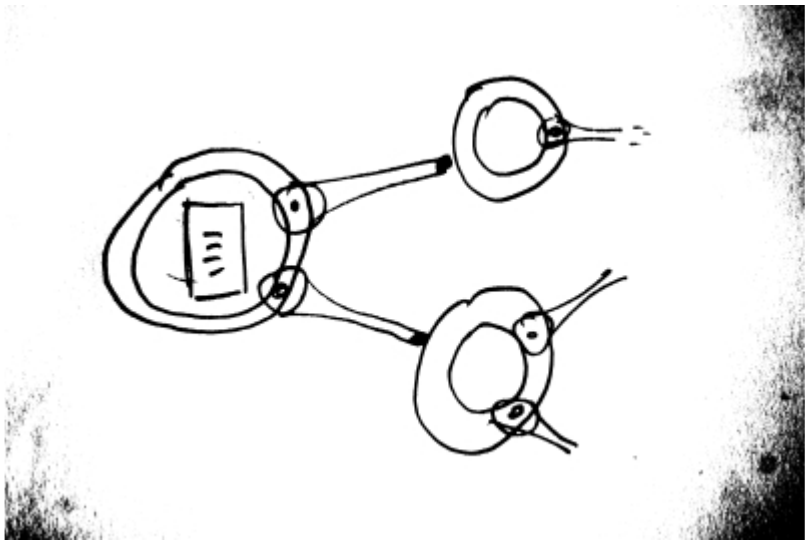
Then time passes ...



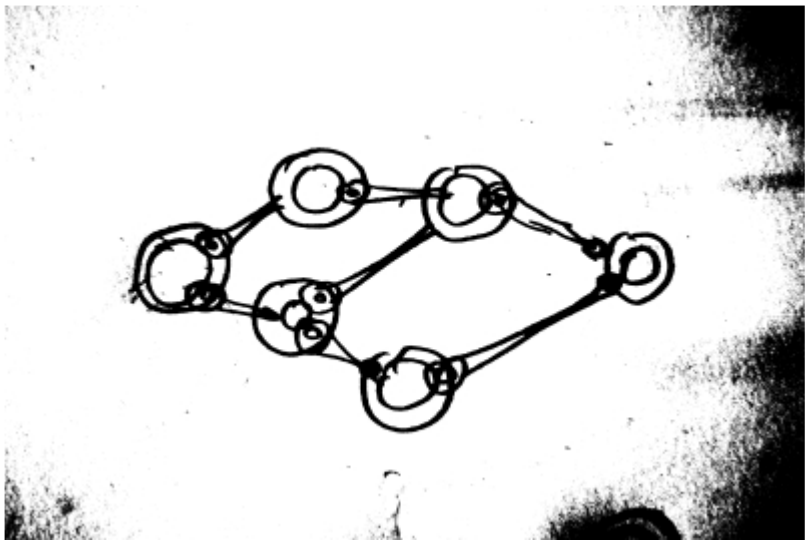
And you have now a so large amount of code that you can't see anything from top of the hill without a large amount of documentation. Top of the hill doesn't talk much as the bottom. Your have no guarantee that your code is reusable. You may apply a standard unit testing method to ensure quality of your code. Who have written those test ? Are you sure that they reach their target ? You just apply a exclusive top-down approach at micro scale.



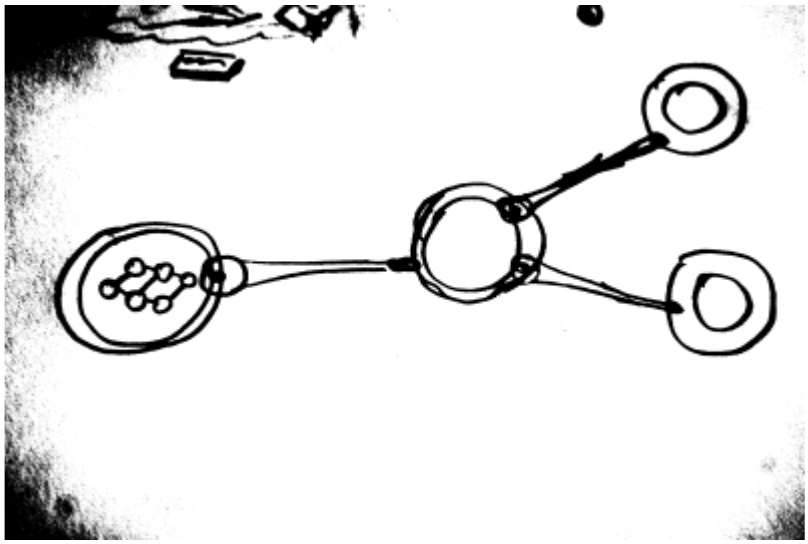
With Cameleon, you start, with your team, to write a page of code.



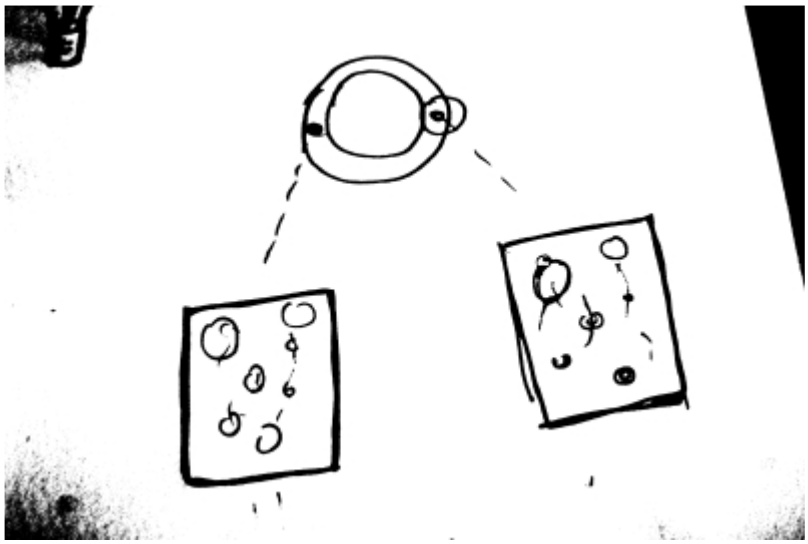
And you integrate this little program into the Cameleon language with dictionary development kit. You get an operator directly integrated to your dictionary of operators.



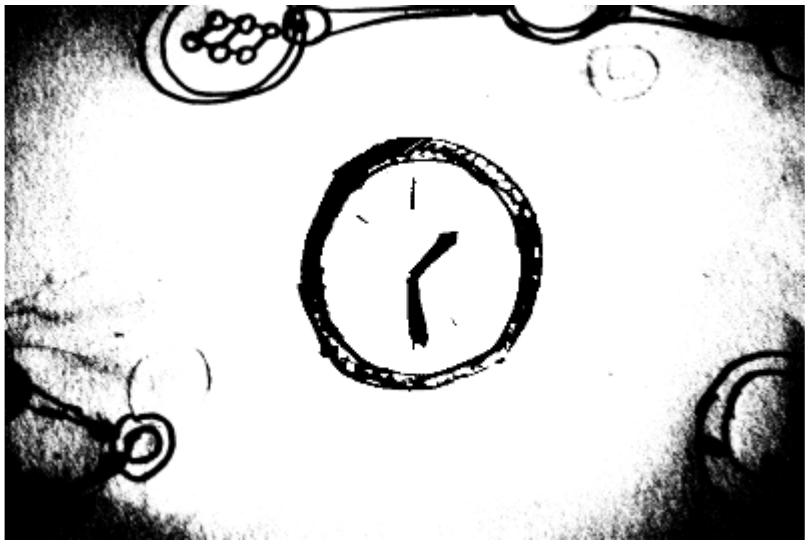
You produce more & more core, you produce more & more operators.



You decompose your problem into sub-problems. Thanks to the language simplicity, this is readable. You can also define an other scale linked to your problem complexity. You have produce severals reusable operators. Unit test & regression test are autonomous thanks to our development kit quality features.

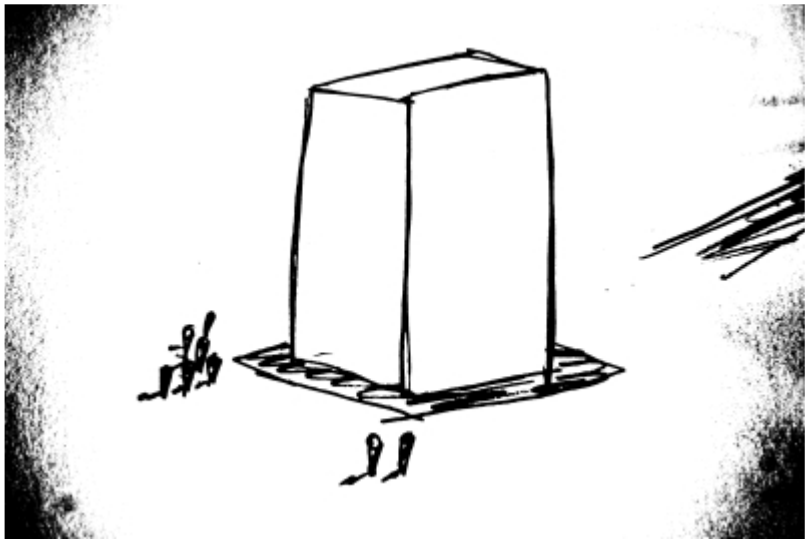


The Data process is viewable & editable via Cameleon controllers.

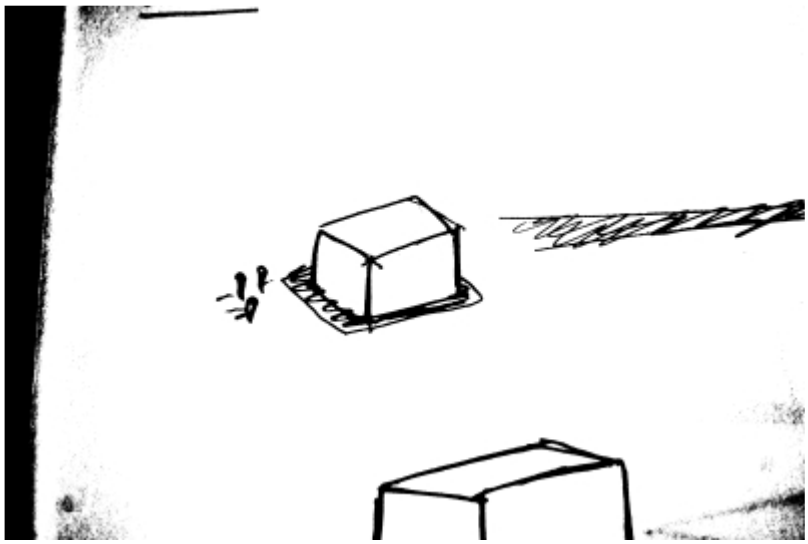


Using Cameleon tools, you go faster, more readable & more sharable!

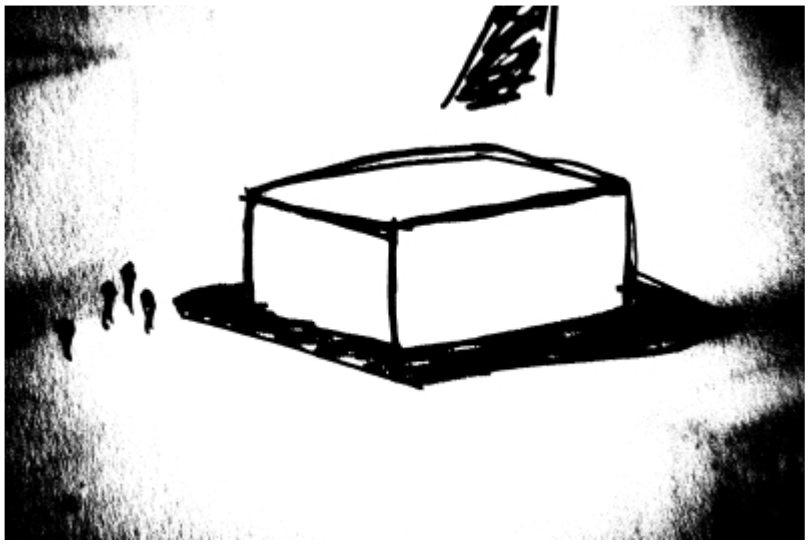
Community vision



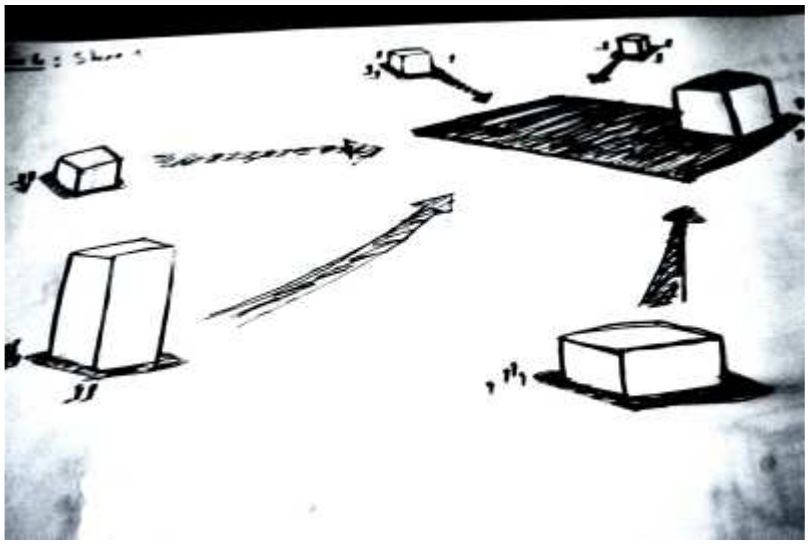
In a complex distributed world, complex problems needs more & more ... from various & heterogenous domain. Or Today, domains are confining in theire own environment ...



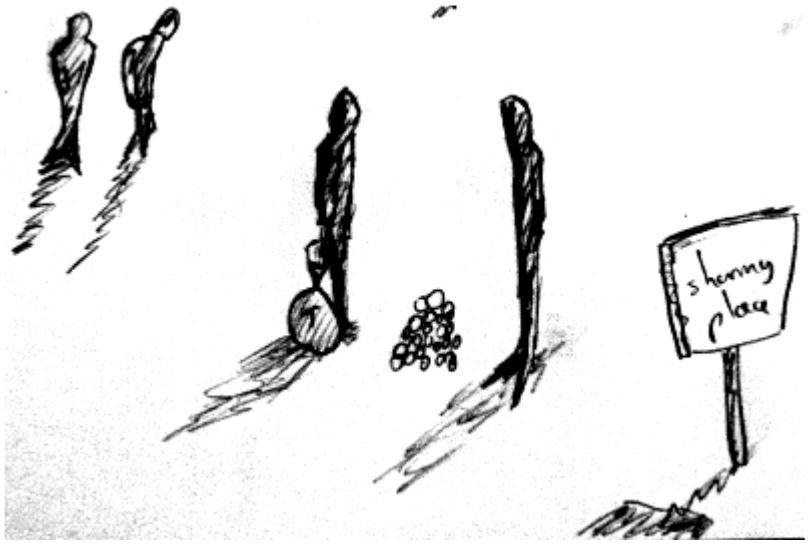
In a complex distributed world, complex problems needs more & more ... from various & heterogenous domain. ... they don't talk to each others.



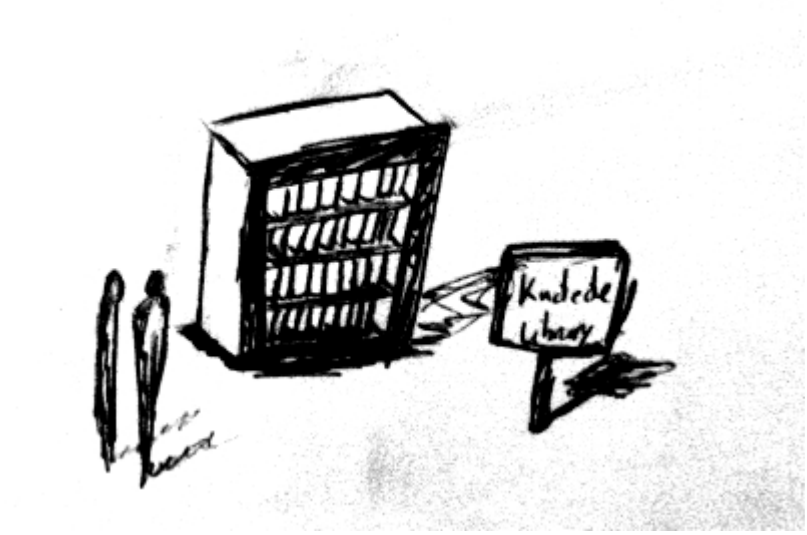
Each domain is located in his own home, alone ! They use their own methodes, tools & functional language. We are in an unintegrated world!



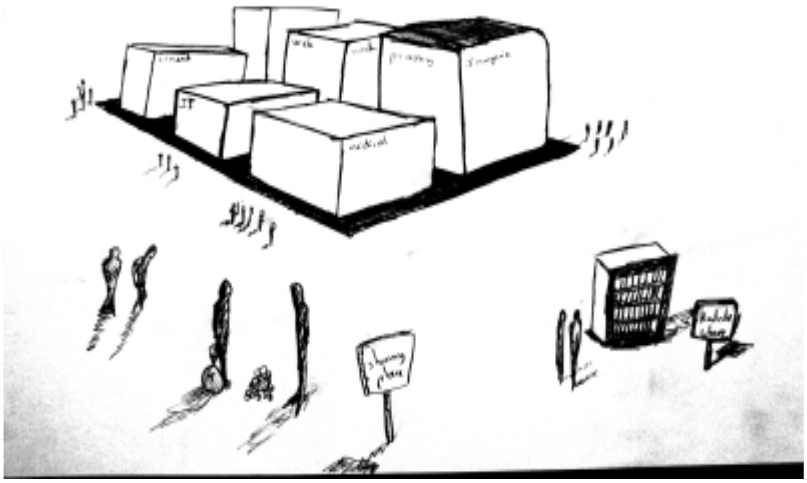
Our Cameleon environment & development kit aims to help various domains work & talk together with a simple, efficient & strong unified language: Cameleon.



It helps you to change your technical-side into a technico-functionnal side where developers & designers work together talking the same language. With the help of ou www.shinoe.org plateform you can share your work to the community. It's a know-how community.

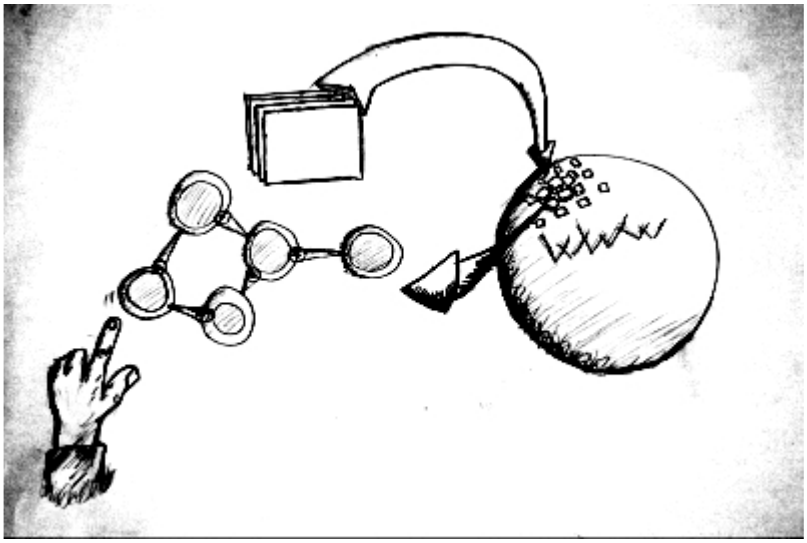


You can share your ideas with others. You can learn from the community. This is a knowledge community.



It makes a community of knowledge & know-how. To use your tools, consumers have just to makes part of the community & use the Cameleon language. Today, we are proud to present you a beta version of the language with a first integrated domain: image processing & analysis. Soon, we will integrate video processing & analysis, a web aggregator tools & much more with the help of our growing community.

The end



Our vision is a world were people capitalize & share their intelligence through an unified functionnal language.

About

Shinoe

Since january 11 2009, Vincent Tariel & Olivier Cugnon de Sévricourt have worked together to develop a language based on new programming paradigms.

Accessibility, efficiency & simplify complexity are some of our goals.

From their collaboration emerged a vision crystallized in their build in progress company: Shinoe.

Shinoe aims to streamline cross-domain interactions by providing software solutions with high added value. We build free software & provide high level software consulting.

Philosophy & Ambition

Our goal is to re-invent programming language & to radically change how to design and build applications. We want to open this activity to all trades, in full independence. Cameleon acts as a glue integrating many different trades areas. It is as much a tool, a language, a method.

We propose a different way of approaching problems and tasks. We propose a new way of thinking. The method is based on finding the right scale. In all areas, it is important to divide the roles between the different scales. The lowest level is no less important than the highest level simply because it sees greater. The highest level is no less important simply because the lowest scale has the ability to feel problems and basic needs. Both must coexist in harmony. The good scale choice allows, ultimately, a better labor division and the scalable developments emergence.

Support team!

You can contact our support team around our forum, pivotal or e-mail adresses.

What's NeXt ?!

Now we are working as hard as we can to deliver an open source deliverable (in our terms, state of art piece of code) cameleon version under the Cecill licence. Currently, this software is under a freeware licence strickly frameworked by ...

We are looking for hands & minds!

We are looking technical friends, phylosophical brothers & work twins to build our best, efficient & must have software we can.

Beta

The beta is open since the 06th october 2011.

Cameleon language & Cameleon composer are shiped with the Standard Cameleon Dictionary (standard types & generics mechanism). You can use, throught Cameleon, the powerfull Population Dicitonary, a graphical access to image processing & analysis.

Licence

Cameleon Language is an open-source software designed by Olivier Cugnon de Sévricourt & V. Tariel under the CeCill licence. Whatever the use, you must cite this document in your related publication.

Limitation

Cameleon beta version including the library Population is an unfinished work. You must create an account on our community plateform, shinoe.org in order to use the Cameleon language.

Please contact us and give us feedback at team [at] shinoe [dot] org

Dictionaries

With Cameleon, you can load & install severals dictionaries to extand cameleon abilities.

For instance, Population is a high performance image processing tool kit shiped as a Cameleon Dictionary. After your account creation on the shinoe plate-forme, you can browse shinoe dictionaries repository by clicking on the dictionary button. This is the place where dictionaries will be shiped & share to our community.

Find Population, download it and install the dictionary using the "install dictionary" feature in Cameleon Composer. See install a new dictionary on the full documentation.

First Release & First Source Release

We will release Cameleon as soon as we can. We hope our beta process can help us finish a great piece of software.

Before releasing the sources, we will work as hard as we can to produce state of art code with a full documentation & a detailed book.

In the near future, our community members will be able to share their work throw the platform. Nowadays, if you want to share your work, or share your intentions, simply contact our team.

First Step

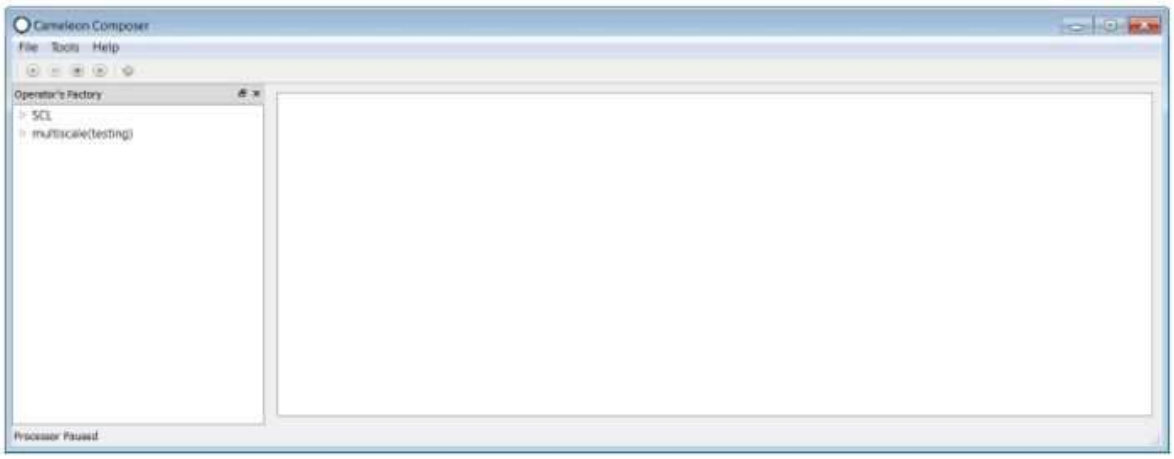
This tutorial aims to give you basics concepts to explore the Cameleon ecosystem & build easily high level applications.

Time to proceed this tutorial: 15 min.

First Launch

At first launch, you will have to accept the license.

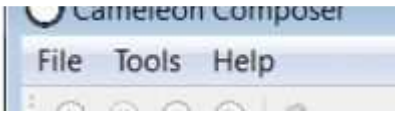
After this validation (we are sure that you do:), the workspace window appears. You're now in Cameleon Composer:



Workspace

The workspace is divided in 4 parts:

- The Menu bar



- The player bar



- The operator's factory



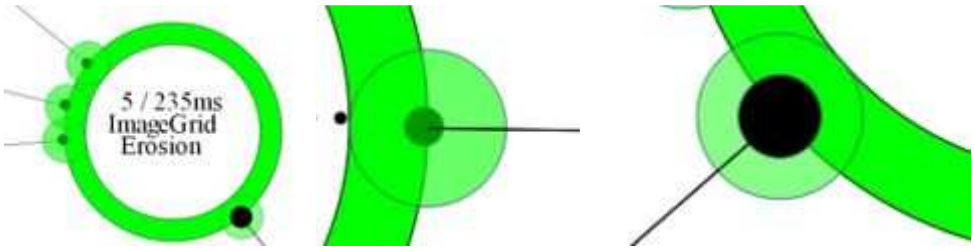
- And, finally, the graph editor (big white rectangle on the first image).

Basics

Start with some definitions.

operator & dictionary

Like a mathematical function, an operator, f, produces output data, y, from input data, x, as follows y = f(x),



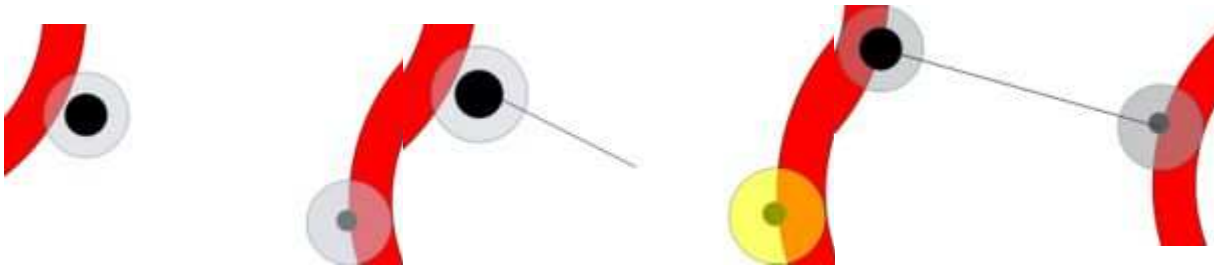
an operator ***input connector*** ***output connector***

An operator (big center circle) has some input connector containing input data (little circles including little black circles) and some output connector containing output data (little circles including big black circles).

A dictionary is a set of operators

connection

The connection is the link done between output plug and input plug for compatible data.

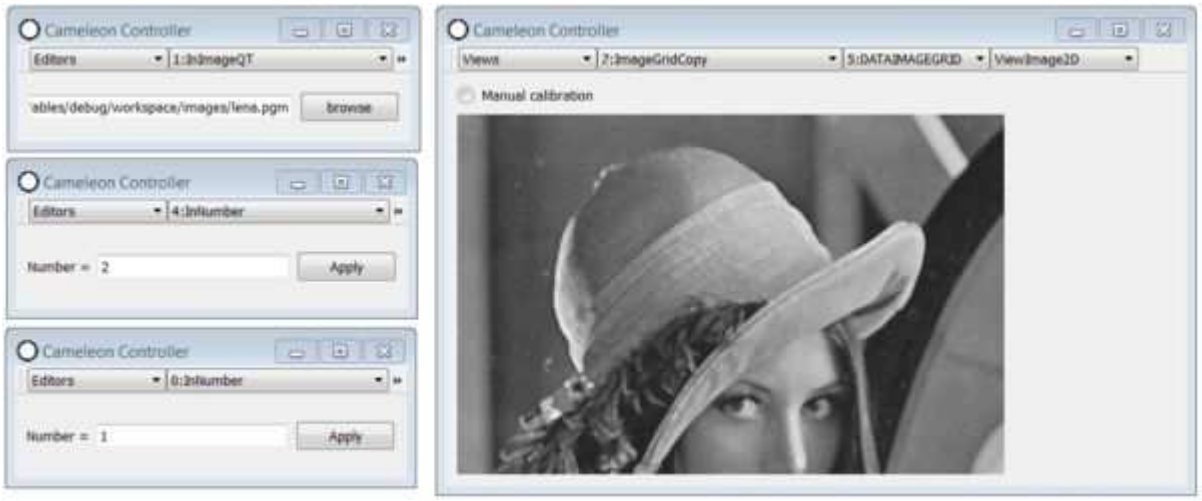


drag to connect choose a compatible connector drag on your choice to connect

editor, viewer & controller

Editors allow you to visually load/save data and viewers allow you to visualize data.

A controller is a window allowing editor or viewer selection.



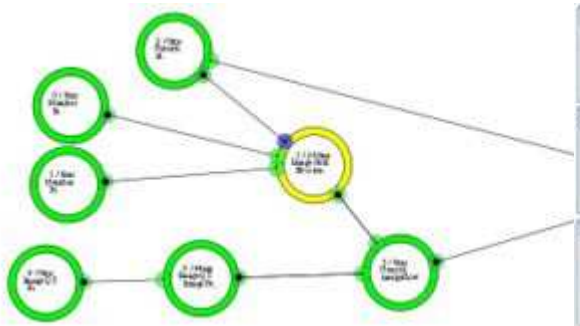
dictionary

A dictionary is a set of datas, operators, viewers & editors packaged to be integrate to cameleon.

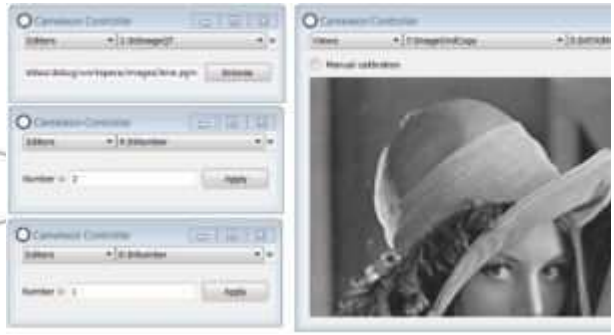


composition & composer

A composition is a set of connected operators (process part) & a set of selected editors & viewers (GUI part) to manage process.



a composition: connected operators



controllers: data editors & viewers

Hello world!

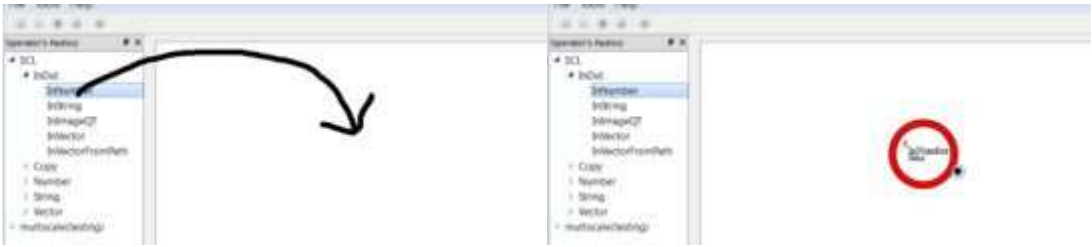
Now cameleon is launched on your system & we will make him say hello to your world following this video tutorial.

A simple addition

The goal of this part is to make a simple number addition.

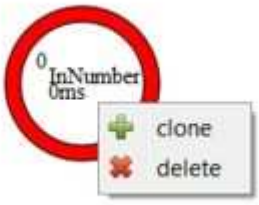
First Operator

To create a new operator, expand the SCD entry in the operator factory, then expand the inOut entry. Select "inNumber" & make a drag & drop of the operator name to the graph editor.



A new operator as been created! Easy !

You can move this operator by drag & drop it. You can delete it or clone it by a simple click on the operator context menu:



click right on the operator then select "delete" or "clone"

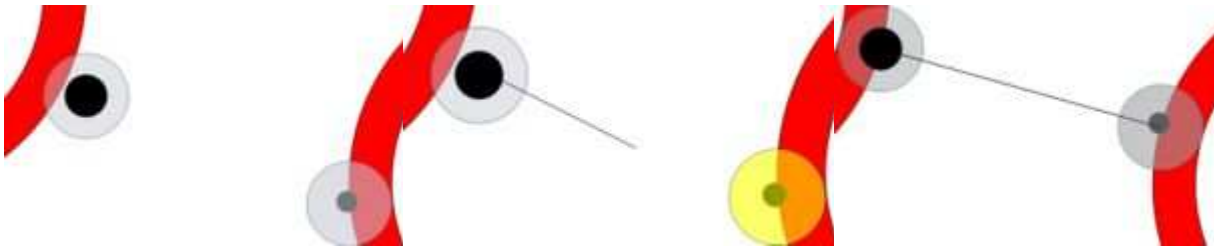
Now, create 3 operators:

- 2 inputNumber
- 1 addNumber

It should looks like this:

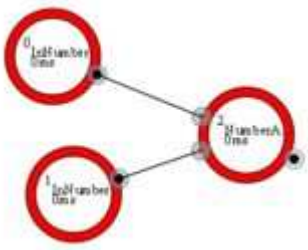


First Connections



To connect two operators, simply click on one connector center & drag it to a compatible connector (yellow circles).

Connect output of each InNumber operator to one input of the addNumber operator. It should looks like this:



Manage the view: move & zoom

The view can be moved by pressing on the CTRL button & by using a simple drag & drop.

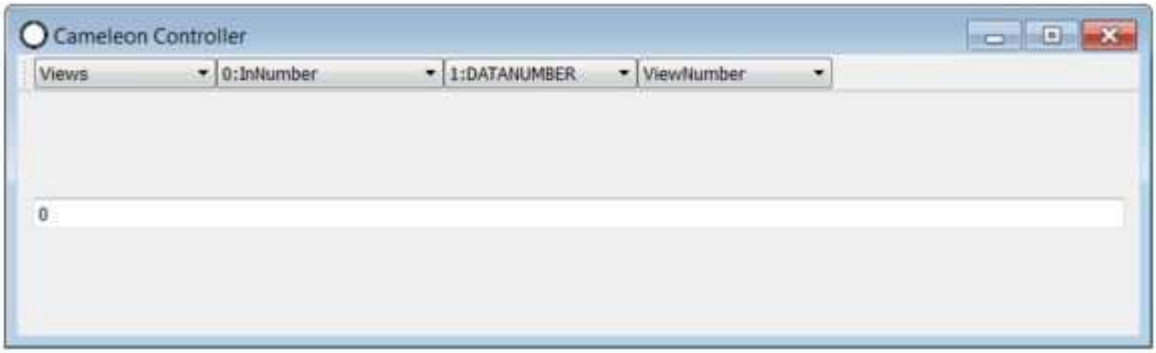
The view can be zoomed by pressing on the CTRL button & by using the mouse wheel.

Data Editor & Viewer

In the menu bar, click on tools and click on "new controller" feature. A new windows appears.

This windows is devided in two part:

- editor/viewer selector:
  - you can select the type (view, editor or operatorGraphics),
  - you can select the operator,
  - the connector/data to display/modify and, finally,
  - you can select the type.
- editor/viewer widget space: this is the place where you will see editors & viewers.



Select editor and choose one of the inNumber operator, like this:

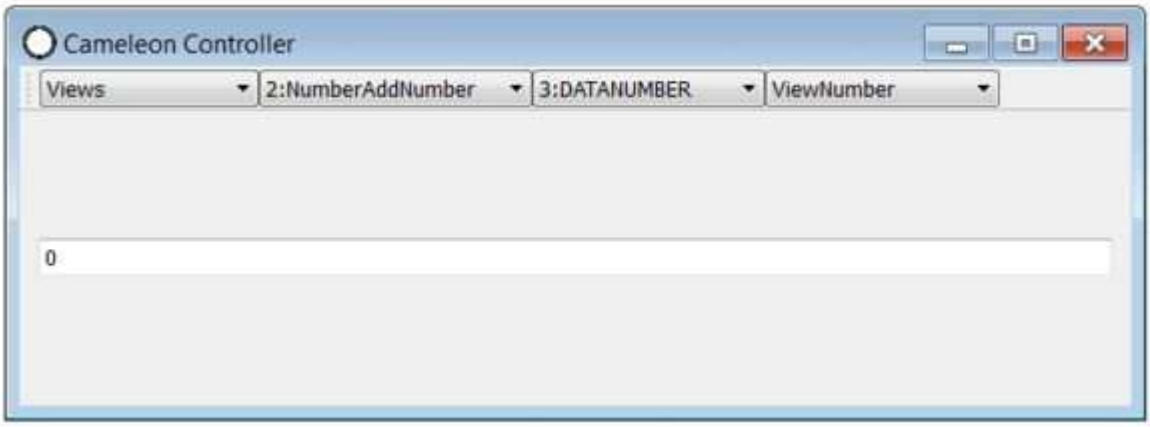


Create an other controller & select the other in number editor, like this:

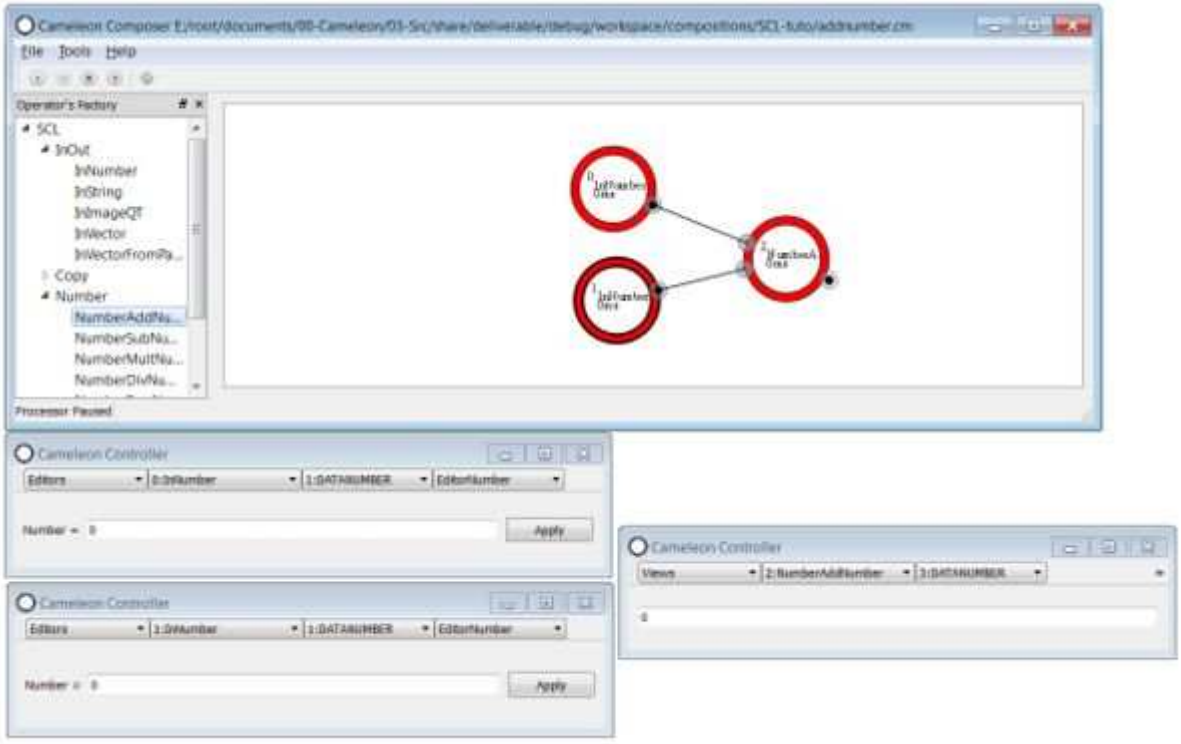


You can create as much controllers as you need.

Create an other controller & select a view on the addNumber operator's output (3:DATANUMBER), like this:



See the whole result:



Launch your beautiful work

On the player bar, click on play and see the result.

A click on "Apply" button on editors update the corresponding data on the process part. You can play around to see a good, simple addition.

Save, Build & launch your work as a standalone application

In the file menu, save your work. Choose myGreatApp.cm for your file's name.

In the "tools" menu, select "build app".

Select your destination directory and press "build". For example, we create a "myGreatApp" directory on the desktop.

Now you can go to this directory, and click on the "myGreatApp" icon.

See Also

- Cameleon tutorials, list of current available tutorials
- Glossary, list of available Cameleon dictionaries
- Documentation, Cameleon IDE full documentation
- Cameleon theory, to know how works the core system

Documentation

Introduction

This documentation aims to discribe all Cameleon IDE features.

Features list



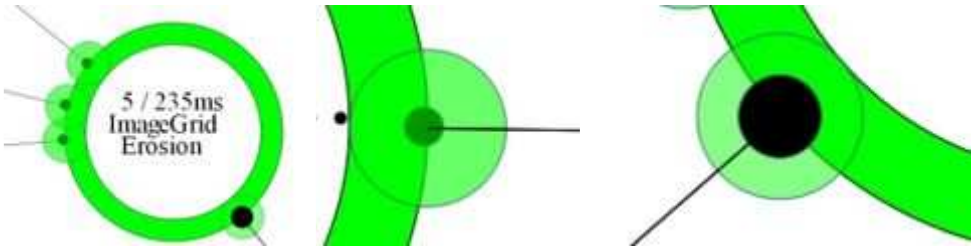
- Interpreter, a Cameleon Language interpreter with:
  - *Real Time Updater*: you update your composition, the interpreter takes care of your change during the execution,
  - *Dictionaries installer:* you install 3rd party dictionary integrated thanks to Cameleon DDK,
- Composer, a graph editor with:
  - *Data process Composer:* you code Cameleon program (named composition) with simple drag & drop,
  - *Real Time Processor:* you see the inner execution of your program in real time,
- Controller, a GUI editor with:
  - *Data Controller:* you create GUI including data editors/viewers with simple drag & drop,
  - *Real Time GUI:* you see & manipulate data in real time,
- Builder, an application builder with:
  - *Standalone Application Builder:* you build & deploy your application with a single click,
- Dictionaries available now on shinoe plateform:
  - *Standard Cameleon Dictionary:* a dictionary dedicated to number, boolean, string & table data,
  - *Generics Dictionary:* a dictionary dedicated to execution loop pattern & conditional execution,
  - *Population Dictionary:* a dictionary dedicated to image processing & analysis.

Basics

Start with some definitions.

operator & dictionary

Like a mathematical function, an operator, f, produces output data, y, from input data, x, as follows y = f(x),



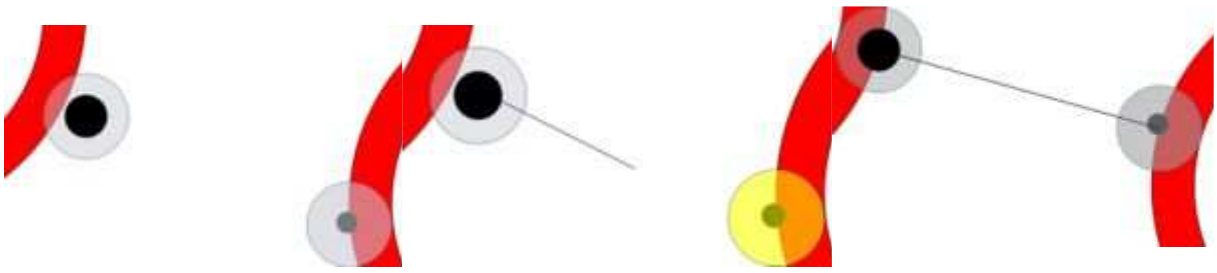
an operator input connector output connector

An operator (big center circle) has some input connector containing input data (little circles including little black circles) and some output connector containing output data (little circles including big black circles).

A dictionary is a set of operators

connection

The connection is the link done between output plug and input plug for compatible data.

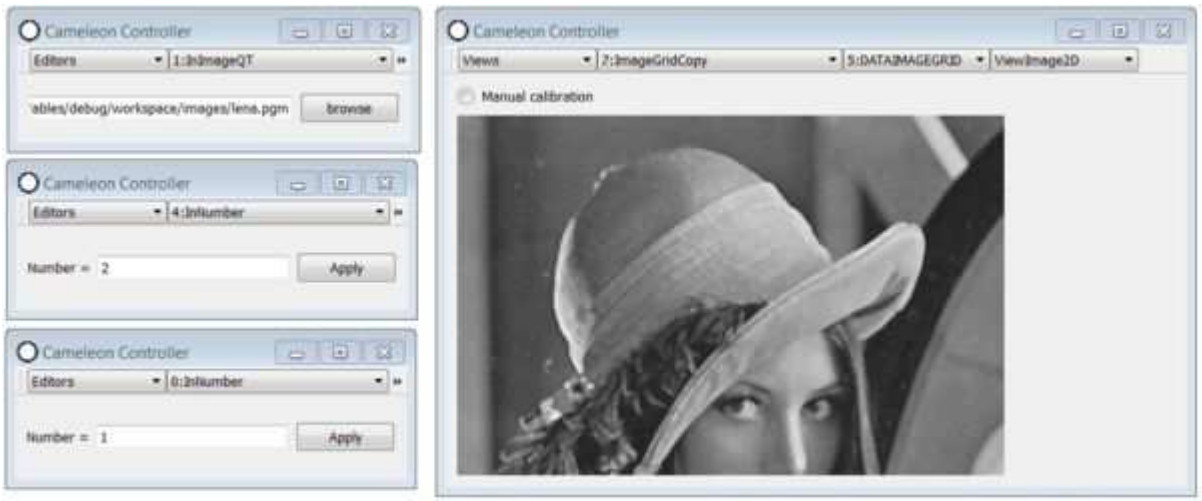


drag to connect choose a compatible connector drag on your choice to connect

editor, viewer & controller

Editors allow you to visually load/save data and viewers allow you to visualize data.

A controller is a window allowing editor or viewer selection.



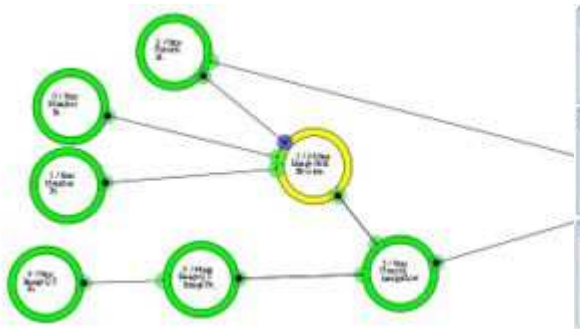
dictionary

A dictionary is a set of datas, operators, viewers & editors packaged to be integrate to cameleon.

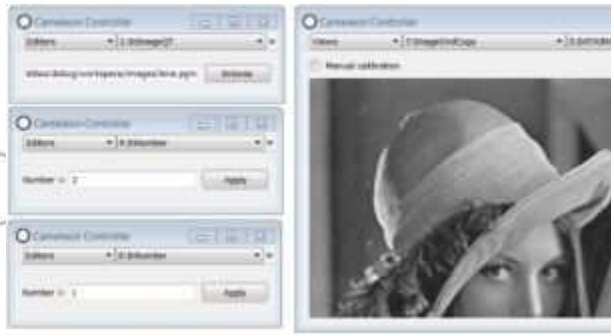


composition & composer

A composition is a set of connected operators (process part) & a set of selected editors & viewers (GUI part) to manage process.



a composition: connected operators



controllers: data editors & viewers

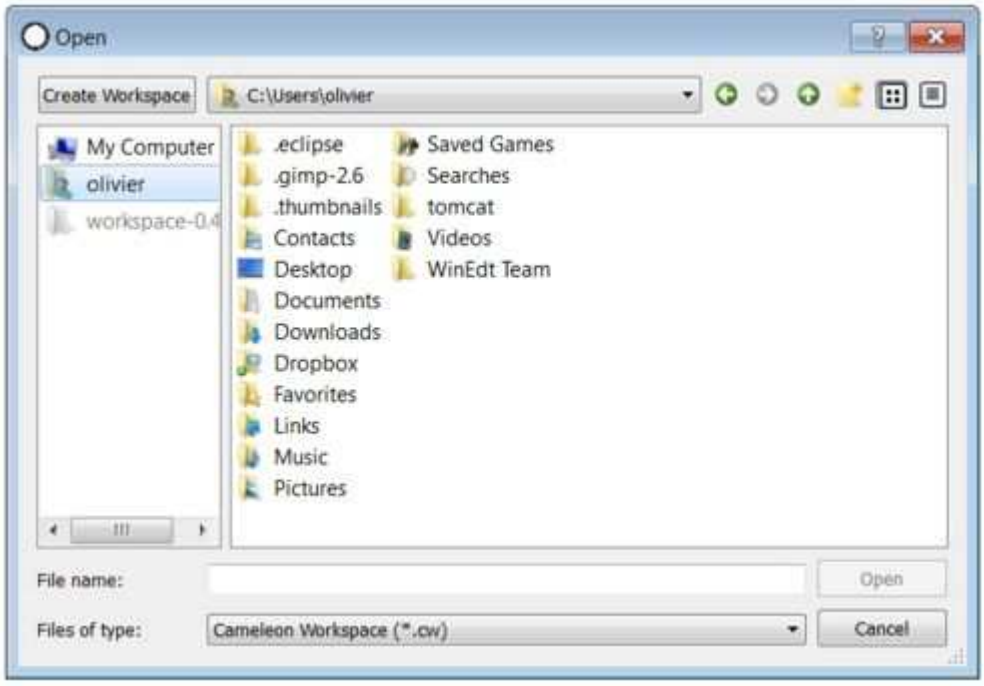
Workspace

The workspace is the directory tree on your computer created & managed by Cameleon IDE to take care of temporary files, extensions, dictionaries, versions management & so on. The workspace entry point is a *.cw file generated by the workspace creation process. This physical workspace is the place where composer will work after the *.cw file load.

Creation

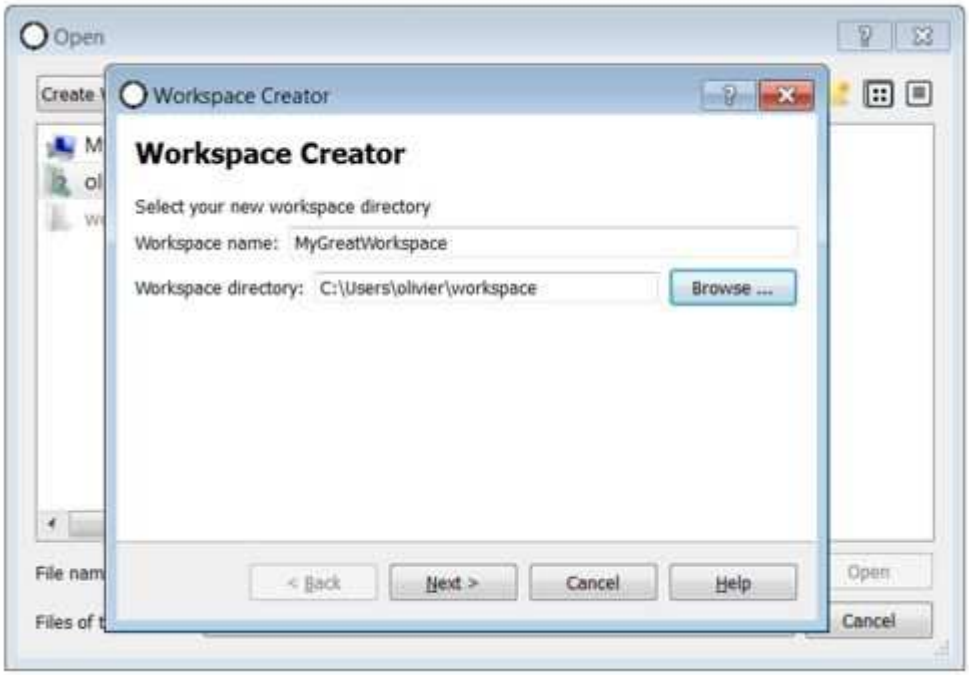
At first launch, you will have to accept the licence.

After this validation, the worspace selector windows appears.



To create a new workspace (you currently haven't one), clic on the top left button named "create workspace".

Select your workspace name and your destination directory.



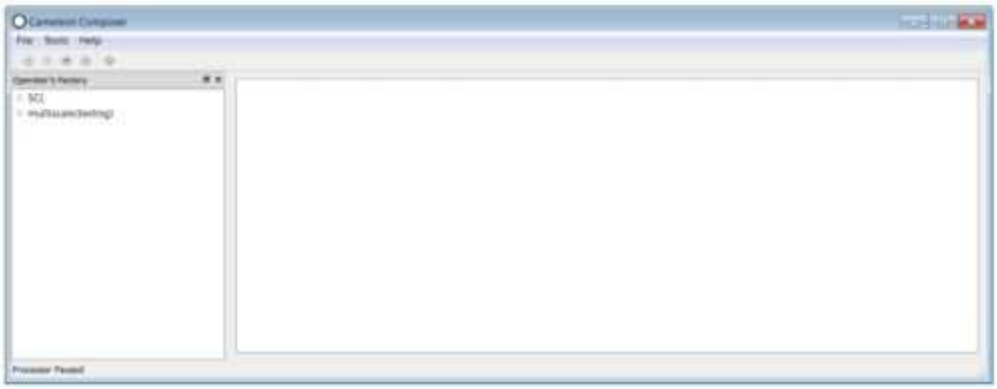
Then press next.

On the next screen press on build. When it's finished press "finish".



Select & Launch

On the workspace selector window, find & select your workspace *.cw file (at root directory of your destination directory) and clic on the open button. Your now in Cameleon Composer:



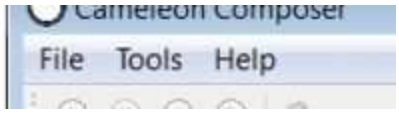
Composer

Workspace

The composer workspace is related to the *.cw file in the workspace directory tree. The *.cw file store all informations needed to load the workspace specifics tools (dictionary, compositions).

The Composer workspace is divided in four parts:

- The Menu bar, gives access to management features



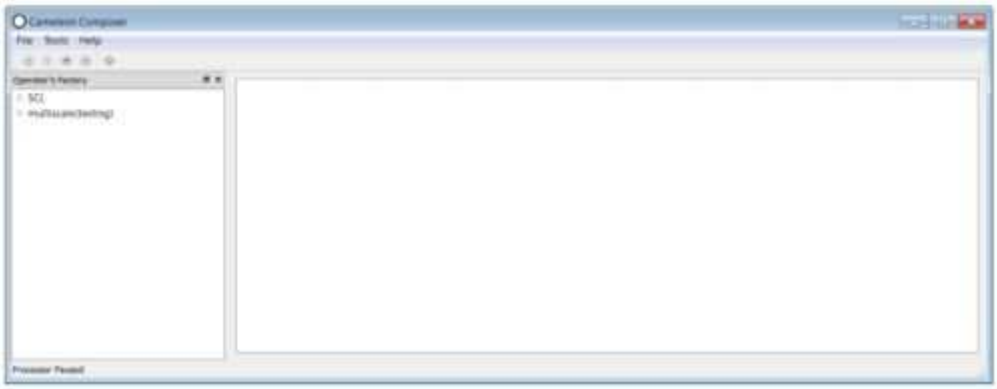
- The player bar, aims to drive compositions



- The operator's factory, gives you access to operator creation



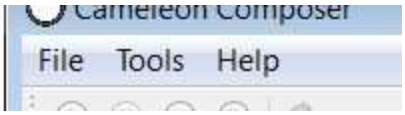
- And, finally, the graph editor (big white rectangle on the following image), gives you access to operator connection, operator creation & operator deletion



Menu Bar

The menu bar is divided in three sub menus:

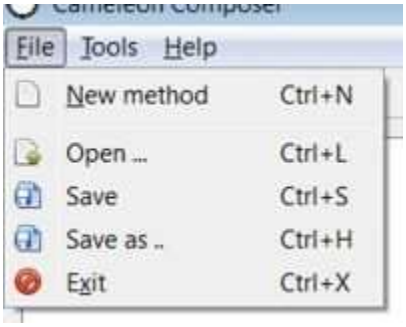
- *File*: manage composition file (save, open ...)
- *Tools*: manage dictionary integration (install, update) & app builder
- *Help*: gives you access to our community



File Menu

The file menu gives you access to five features:

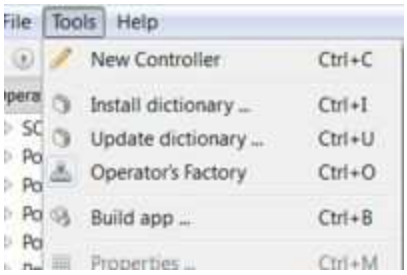
- *New Composition*: Create a new composition from here
- *Open ...*: Open a saved composition from here (warning: all unsaved data are lost)
- *Save*: Save a composition from here
- *Save as ...*: Save a composition to an other place from here
- *Exit*: Exit from here (warning: all unsaved data are lost)



Tools Menu

The tools menu gives you access to six features:

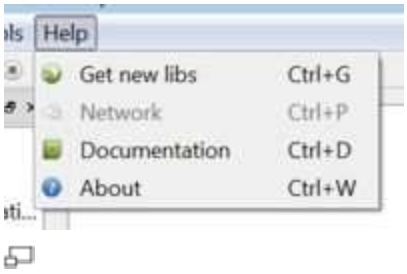
- *New Controller*: Open a new controller
- *Install Dictionary ...*: Install a dictionary
- *Update Dictionary*: Update an existing dictionary
- *Operator's Factory*: Show/Hide operator's factory (useless)
- *Build App ...*: Build an app for a saved composition
- *Properties ... (coming soon)*:show properties



Help Menu

The help menu gives you access to four features:

- *Get new dictionaries*: get dictionaries from www.shinoe.org
- *Network (coming soon)*: install & update dictionaries directly from Cameleon IDE
- *Documentation*: access to this web site
- *About*: access to Cameleon IDE informations

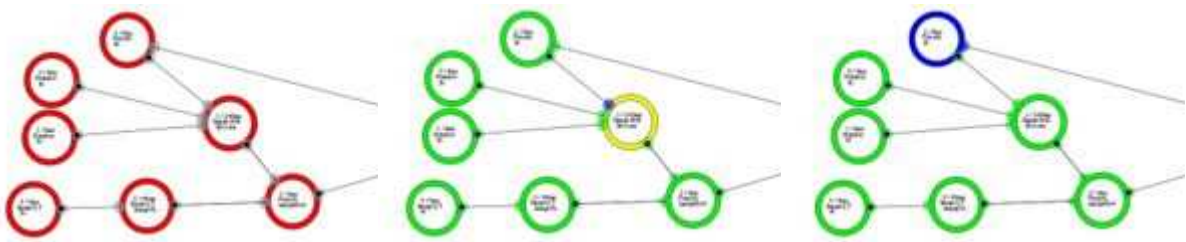


Player Bar



The player bar allow you to control the Cameleon interpreter:

- *Run*: to launch the composition execution
- *Pause*: to pause the execution
- *Stop*: to stop the execution
- *Next step*: in pause mode, allow you to process one next operator



stopped composition running composition paused composition

Operator's Factory

The operators factory gives a view on all operators available on the workspace for creation.

You can create an operator by a simple drag from an operator name in the operator's factory following of a drop into the Graph Editor.

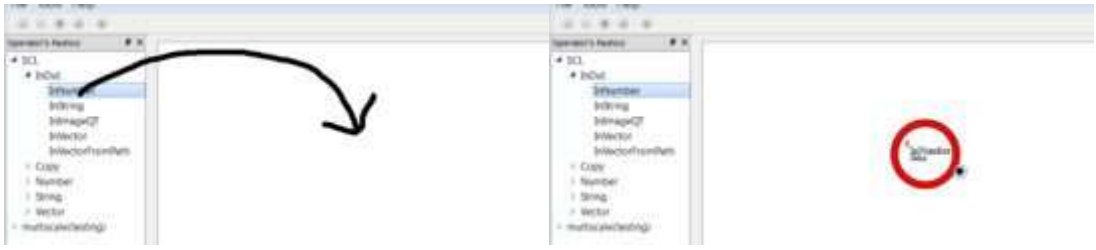


operator's factory

Graph Editor

Create & delete Operator

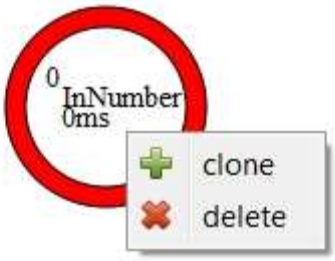
A simple drag & drop must be done to create an operator.



drag drop

An operator context menu is available by clicking right on an operator. A context menu appear with two construction features:

- delete: to delete the operator
- clone: to clone the operator



operator context menu

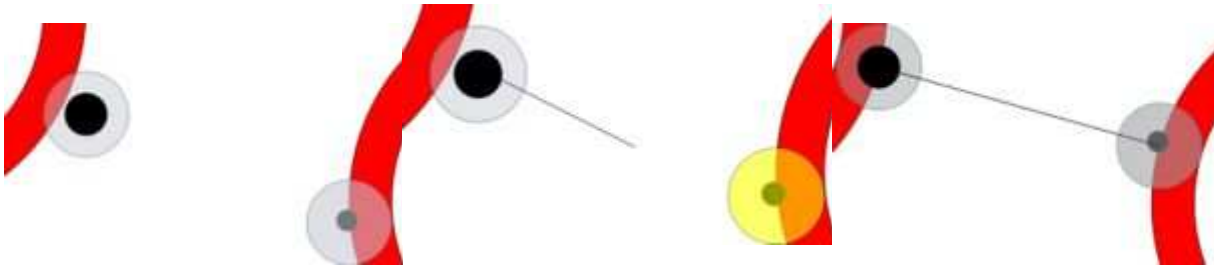
The delete menu is not available when process run.

Move an Operator

Operator can be moved by drag & dropping them.

Connect & disconnect Operator

To connect two operators, just drag from the center of a connector. At this point, compatible connectors must turned to yellow color. Continue to drag to a compatible connector to connect.



two connector, start a drag | two compatible operators, drag to a compatible operator | drop to connect

A connector context menu is available by clicking right on a connector. It allow:

- *disconnection*: to disconnect the connector
- *add*: to add an other connector to the operator (when the operator allow this feature)



connector without add featureconnector with add feature

The disconnect menu is not available when a linked operator is running.

Manage the view: move & zoom

The view can be moved by pressing on the CTRL button & by using a simple drag & drop.

The view can be zoomed by pressing on the CTRL button & by using the mouse wheel.

Clear & properties

By clicking on the Graph Editor (into the white void), you will have access to the Graph Editor context menu showing those features:

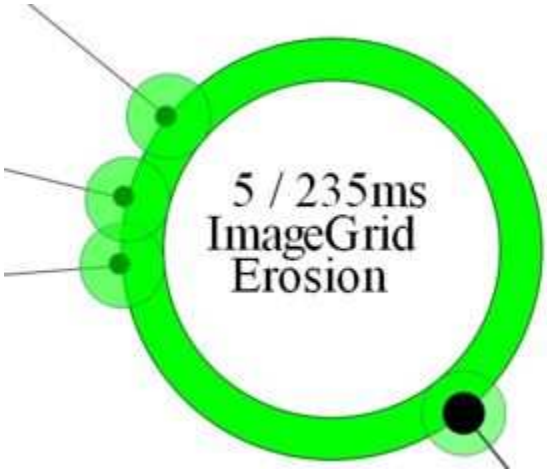
- *clear*: delete all operators & all controllers created
- *properties (available soon):* gives an access to the Graph Editor properties screen.



Operator

An operator (big center circle) has some input connector containing input data (little circles including little black circles) and some output connector containing output data (little circles including big black circles). Some information are exposed on the big center circle:

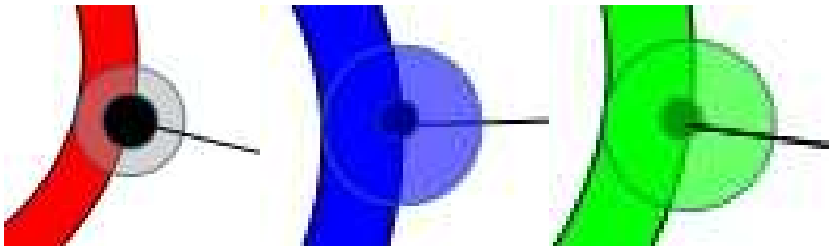
- *id*: the operator id, usefull when you have to choose a data controller
- *based data type*: data type processed by this operator
- *name*: operator functional name



Connector Colors & statuses

A connector has three possible statuses:

- *empty (grey)*: their is no data stored by the connector
- *new (blue)*: a new data (not consumed by the operator) is stored by the connector
- *old (green)*: an old data (already consumed by the operator) is stored by the connector



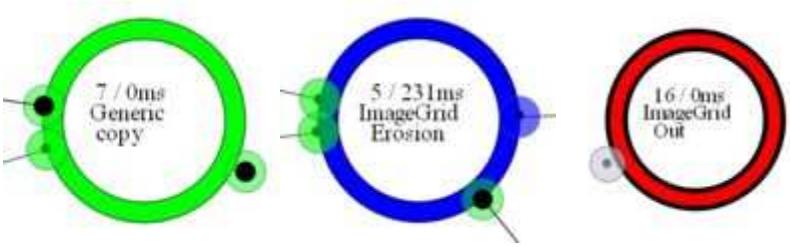
empty new old

When you start a drag to connect a connector, a yellow compatibility marker is placed on compatible connectors so you can connect your dragged connector to those connectors.

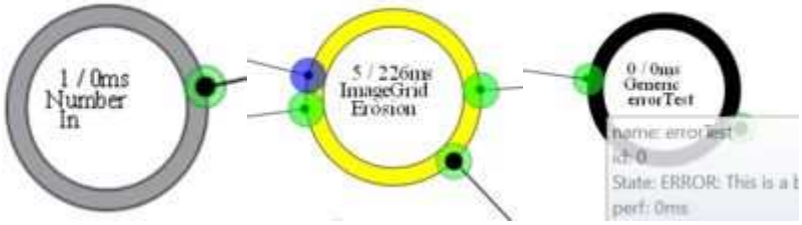
Operator Colors & statuses

A operator has four possible statuses:

- *Has been run (green)*: the operator has been run, input data has been consumed to produce output data
- *Runnable (blue)*: the operator is runnable, the launch condition is reached
- *Not Runnable (red)*: the operator is not runnable, the launch condition is not reached
- *Waiting (grey)*: the operator is waiting is turn to run
- *Running (yellow)*: the operator is running
- *Error (black)*: an error occured, see information tooltip



have been run runnable not runnable



not runnable not runnable error

Information tooltips

Some operator & connector information can be grab with the help of tooltips.

operator

The operator's tooltip is available by passing the mouse cursor hoover an operator. It show you some informations:

- *name*: operator's name
- *id*: operator's id
- *State*: in error case, you will find here the reason
- *perf*: time to proceed this operator in previous iteration
- *usage*: usage description

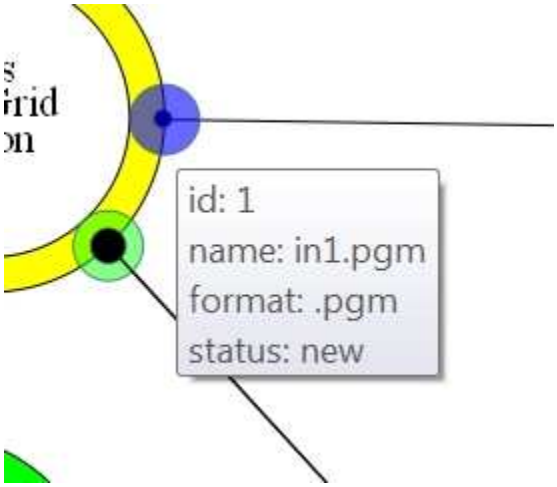




connector

The connector's tooltip is available by passing the mouse cursor hoover a connector. It show you some informations:

- *id*: connector's id
- *name*: functional name
- *format*: data format
- *status*: empty, new or old



Invisible connector

Most operators need input to produce output data. Some of them grab their inputs or outputs out of the system. In this case, no input or output connector is shown on the graph editor. Those operator (without input or output) are, most of the time, coupled with a data editor to set the data.





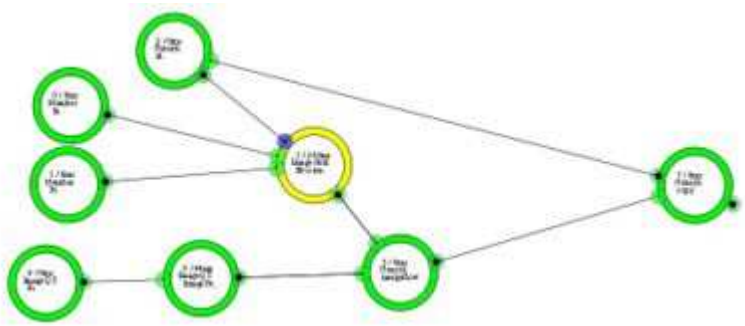
Execution condition

See article.

description cas standard

Composition

A composition is a set of connected operators & a set of data editors & viewers.



Controller

Controllers are made to interacte with the data process: visualize & manipulate datas.



Selector bar

The selector bar is divided in four part:

- *controller type selection*: View, editor or operator graphic
- *operator selection*: Choose an operator from the graph editor. The first number is the operator id follow by the operator name.
- *connector selection*: Choose the connector. This selection is restricted by operator's connectors. The first number is the connector id followed by the data type.
- *type*: Choose wich editor or viewer you want to use. This selection is restricted by the data type. Sometime, it exists multiple ways to view or edit the same data type.



The selector bar

View

A view allow you to visualize data. See glossary for more informations.

Here are some examples:



image viewer

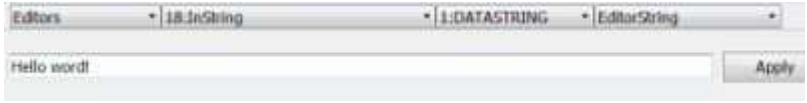
Editor

An editor allow you to manipulate data. See glossary for more informations.

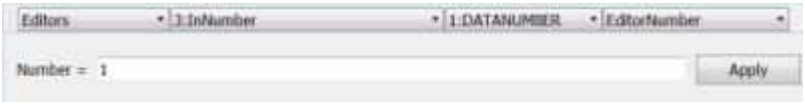
Here are some examples:



file editor



string editor



number editor

Operator Graphic

An Operator Graphic is as much an operator as an editor. The process is fully manual. Input data is shown to the user & output data is produce by the user.

The first Operator Graphic example is the marker:

- input is an image
- ouput is markers drawn by user

You will find more information about operator graphics following Watershed tutorial (with markers).

Manage dictionaries

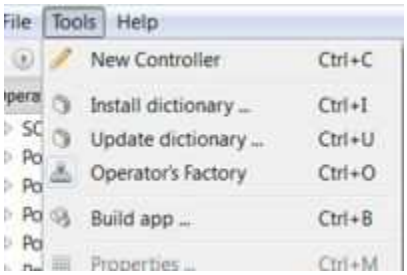
Install a new dictionary

Before any dictionary installation, your operator's factory looks like this:

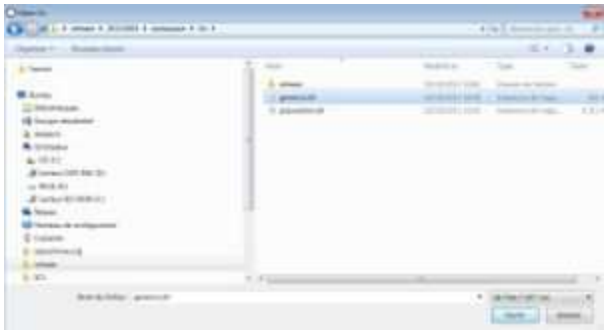


You have only SCD operators available. To extend Cameleon features by adding new datas/operators/view and/or editors, you can use the install dictionary features.

On the menu bar, clic on tools, then clic on "install dicitonary".



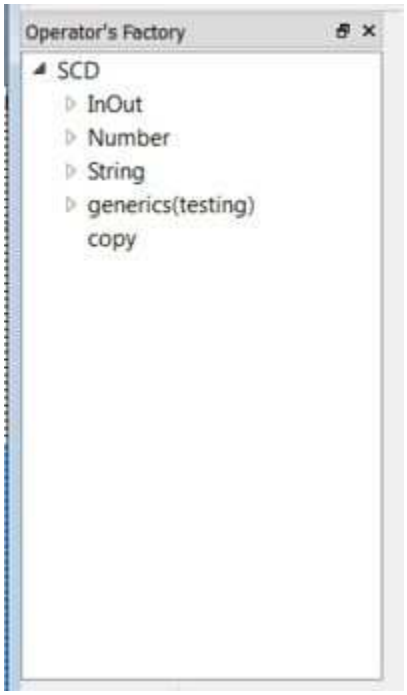
A new window appear. Select the lib you have downloaded following this (.dll file for windows & .so files for linux), then clic on open.



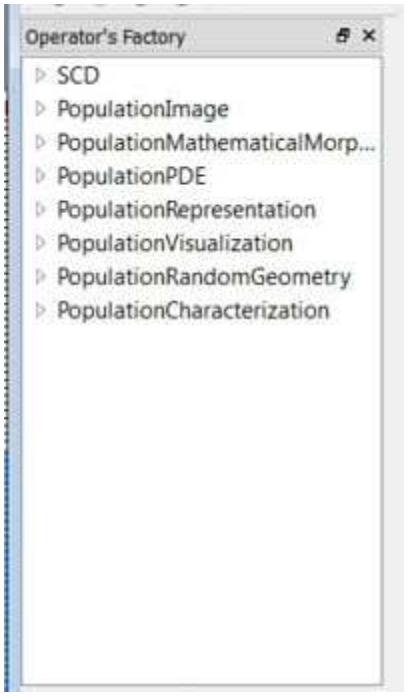
A windows appear showing you the dictionary integration progress.



After generics installation, you have new operator in the SCD dictionary. Generics extend SCD dictionary features:



After population installation, you have 7 new dictionaries. Population documantation is available here [1].



Update an existing dictionary

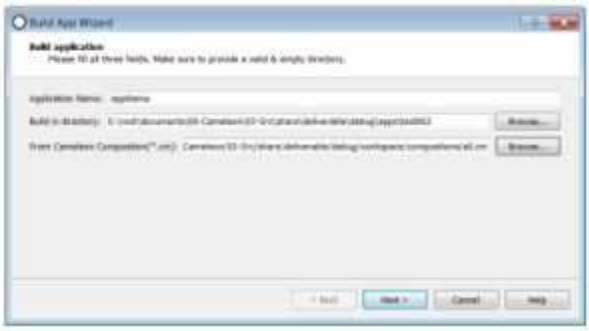
This process is the same process as Install dictionary. You must have already installed the dictionary you update.

Build app

With Cameleon IDE you can build standalone application software from composition.

In the "tools" menu, select "build app".

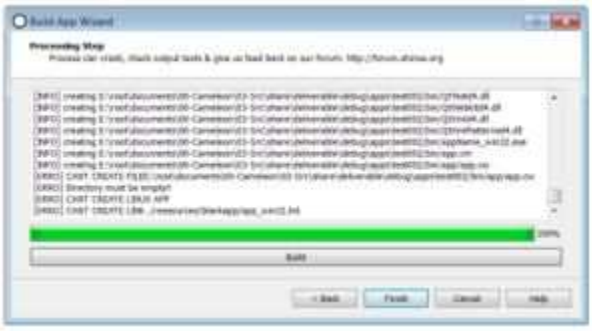
Select you name, your composition (*.cm) & destination directory and press "build". For example, we create a "myGreatApp" directory on the desktop.



Press on the build button.



When the process is finished, you can go to this directory, and click on the "myGreatApp" icon.



Here are some applications examples:





See Also

- Cameleon tutorials, list of current available tutorials
- Glossary, list of available Cameleon dictionaries
- Cameleon theory, to know how works the core system

High level bottom-up

Today, programming language & development activities are core components of our systems. Software development approach is generally a top-down which main paradigm is *design by knowledge* mixed with a little frameworked bottom-up (scrum, agile).

Design by evolution [1]

Design by knowledge

Contents

[hide]

- 1 Multi-scale programming
  - 1.1 Top-down
  - 1.2 Bottom-up
    - 1.2.1 Low-level
    - 1.2.2 High-level
  - 1.3 Mix
- 2 Cameleon, a framework to design high-level bottom-up languages
- 3 Some hypothetical use cases
- 4 annexes
  - 4.1 quoted[2]

Multi-scale programming

Software data-process is any process where input and output are data such as recording, analysing, sorting. Because process complexity have grown continuously, it emerged a software development in a multi-scale approach that can be top-down or/and bottom-up and slitted into different activities, requirements, specification, architecture, implementation, testing.

Top-down

The natural thinking of specification is to start with a global overview of the process as an organisation of sub-processes and to iterate this task for these sub-processes until the base level of granularity. This top-down paradigm is often chosen to have a coherent development with a prior specification activity. Popular languages as C#, C++, java are imperative language including object-oriented paradigm. Their approach is more oriented top-down due to the inheritance mechanism from the abstract classes to the concrete classes. This approach does not guarantees :

- *flexibility* : because specifications and, thus decisions, are done at the beginning of the project, it is difficult to make adjustments after without changing the whole structure,
- *modularity* : because of the hierarchy governance from the global to the local, local is not the heart of the development and, thus, its modularity.

Bottom-up

Note, Critique de Feldman

Low-level

Low-level bottom-up states that we do not have a clear idea about the whole software organisation at the project beginning. Basically, specification/design/implementation is done component by component and the global software emerged from theses components organisation. However, most of the time, this development is not orthogonal in the sense that each component depends on the specific formulation of other components. A basic example is a project developed by students.

High-level

High-level bottom-up approach is focus on how to design some elementary bricks allowing to build a class of data-processes based on the composition of these bricks. The dictionary of bricks depends on the computer field. This approach is a real conceptual challenge in order to respect these criteria :

- *transversal* : the dictionary allow the connection with other dictionary,
- *versatile* : the dictionary allows the composition of a wide-range of data-processes with even some of them not expected by the creator,
- *scale* : the brick semantic has to be chosen at the relevant scale, not too small to avoid that the composition includes too many bricks, not too large to avoid a composition with a single brick.

Mix

Nowadays, both paradigms are often mixed in projects. For instance,

- programmers develop a first version of a software in a low-level bottom-up approach. Then, for the next version, they can make the relevant decisions in the top-down approach with a modular and flexible architecture software,
- the specifications is done from a top-level to a middle level following by an architecture design respecting this formulation. Then, for each component at the middle level, programmers codes in a low-level bottom-up paradigm,
- the elementary bricks of a high-level bottom up approach are designed following a top-down approach in an oriented object programming approach.

Cameleon, a framework to design high-level bottom-up languages

Cameleon is the first framework dedicated to the high-level bottom-up approach. Its language contains a new grammar based on a petri net model extension. This extension allows that compositions containing conditional executions and loops stay simple. A programmer can integrate his own bricks defining a new dictionary with a simple registration of C++ classes. Based on the semantic language, we implement :

- a processor to execute at runtime a composition,
- an editor to build a composition and to call the processor.

Some hypothetical use cases

A researcher (ie using the computer as a tool support) has written a collection of algorithms (micro). We name this collection a dictionary. This dictionary aims to extract objects from images and image statistics computing related to these extractions. This dictionary uses its own data structures and has a clean formalism inaccessible to the uninitiated. The integration of this collection in Cameleon is emerging a language (based on complex operations and data) related to the researcher's job (macro). This opens the possibility to communicate effectively on this work (sharing). This also opens up a simple way to develop and calibrate complex application. This integration gives him a second thing: his library (his language, his domain) has now access to existing dictionaries in the cameleon ecosystem.

A designer / graphic designer is that existing computer tools allowing him to set up treatment needed for these creations are formatted by the limited ability to change the treatment unit. It uses Cameleon, providing access to thousands of image processing operators to dial through the language of image processing, new treatments. Withdraws its creation at the composition software, the work is the algorithm.

A journalist familiar with excel and a scripting language. Her job asks her to understand a gigantic mass of information. This informations can be found in videos, photographs and texts. She has this information and create treatments to extract relevant informations. These treatments are isolated unit and have a high heterogeneity. She decides to use Cameleon as this will allow easy combination of treatments that will be carried out for different missions. It is easy to understand what "business intelligence capitalization" means. During many projects, she adapts and develops her toolbox. She also shares "tips & tricks" with the Shinoe community. The library of the researcher of the first example also allows to automate the viewing of videos to create catalogs of videos related to a subject, a character. Another library allows it to connect to the web to retrieve video and images to analyze, always using the first example dictionary.

annexes

quoted[2]

In typical code, once you abstract out the parts which are merely bookkeeping, what's left is much shorter; the higher you build up the language, the less distance you will have to travel from the top down to it. This brings several advantages:

1. By making the language do more of the work, bottom-up design yields programs which are smaller and more agile. A shorter program doesn't have to be divided into so many components, and fewer components means programs which are easier to read or modify. Fewer components also means fewer connections between components, and thus less chance for errors there. As industrial designers strive to reduce the number of moving parts in a machine, experienced Lisp programmers use bottom-up design to reduce the size and complexity of their programs.

2. Bottom-up design promotes code re-use. When you write two or more programs, many of the utilities you wrote for the first program will also be useful in the succeeding ones. Once you've acquired a large substrate of utilities, writing a new program can take only a fraction of the effort it would require if you had to start with raw Lisp.

3. Bottom-up design makes programs easier to read. An instance of this type of abstraction asks the reader to understand a general-purpose operator; an instance of functional abstraction asks the reader to understand a specialpurpose subroutine.

4. Because it causes you always to be on the lookout for patterns in your code, working bottom-up helps to clarify your ideas about the design of your program. If two distant components of a program are similar in form, you'll be led to notice the similarity and perhaps to redesign the program in a simpler way.

TODO WORK

This language is based on a simple scale split principle:

- Micro scale, written language, is effective in solving local problems
- Macro, graphic language, is effective in solving global problems

Because of this split, and unlike the main tools competitors, Cameleon leverages the advantages of both approaches: micro to adress local, macro to adress global. It guaranteed maximum efficiency regardless to the problem scale. The language gives to independent (researchers, engineers and developers), whose computer is the tool preferred medium, a framework for combining work flexibility, usability & accessibility. This graphic language allows to create your own business language and ensure a simple and effective functional communication to your clients. This language and new languages creation method guarantee transparent, accessible and flexible applications constructions. Within minutes, the grammar and vocabulary basics are acquired to address the language and its components.

This language is a petri net extension, a strong & powerfull mathematical theory. See the cameleon theory.

Through these three examples we have highlighted three key features of language Cameleon:

- Easy, open & transparent app building
- Working languages creation
- Default interoperability